PIERNIK · BRANCH SF_STREAMINGCR_COMB · COMMIT AFTER B22C7360A

Consistency fixes in star formation, SN feedback, self-gravitating hydrostatic equilibrium and CR–gas exchange: change log

Base commit **b22c7360a** · Date **29 October 2026** · Prepared for **Vinod V. Pisharody**

This commit collects the fixes made after b22c7360a, while reviewing the tallbox_sf comparison setup and before the 500 Myr production runs C1 (MHD + diffusive CR) and D (MHD + streaming CR). Each fix removes an inconsistency: a formula that differs from the one it is meant to match, a quantity computed on the wrong state, or an equilibrium that is not one. The production runs were built from exactly this tree. Section B gives the validation, section C three problems found in those runs that this commit does *not* change, and section D the state of the runs.

9	**7**	**236 Myr**	**3**
records in 9 files: 4 source, 4 parameter files, 1 script	bug fixes: wrong rates, double SF, non-conservation, a non-equilibrium initial state	of MHD + CR production run time on this tree (C1 134 Myr, D 102 Myr), no NaN, no crash	other problems touched (sf, stratified_box, tallbox) compile and run

BUG FIX wrong results or non-conservation **DIAGNOSTIC** warnings, no effect on results
CONFIG parameter files, scripts **OPEN** found, not changed here

Contents

1. Star formation ran twice in the sf problem · problems/sf/initproblem.F90
2. Free-fall time with the stellar density · star_formation.F90
3. SN kinetic energy on the final cell state · star_formation.F90
4. Seed-star masses use the SF rate · star_formation.F90
5. CR–gas exchange after the floor and the clip · streamingcr_source.F90
6. Column density with self-gravity · hydrostatic.F90
7. Self-gravity: midplane on a face, trapezoid rule · hydrostatic.F90
8. Convergence warnings · hydrostatic.F90
9. Parameter files and script · problem.par ×4, h52paraview.py

Then: **B** validation · **C** open issues found in the production runs · **D** state of the runs.

A. Overview: before, after, motivation

#	BEFORE	AFTER	MOTIVATION
1	sf problem calls `SF` and `init_SF` itself; the core calls them too	Only the core calls them; the problem only adds up the formed mass	Stars formed twice per step, SF initialised twice
2	`32 G ρ + ρ*` inside t_{ff}	`32 G (ρ + ρ*)`	Operator precedence: ρ_* weighted ~7× too much
3	SN momentum from the cell state before mass injection and equalisation	Momentum from the final state, adds exactly the intended E_{kin}	Kinetic energy off by an uncontrolled amount; needed a `max(0,…)` guard
4	Seed mass $\varepsilon \sqrt{G}\, \rho^{3/2}\, \Delta t$	$\varepsilon\, \rho/t_{ff}\, \Delta t$, as in SF	Seeds 1.84× lighter than the SF law they stand for

5	Gas paid for ΔE_c, ΔF_c before the floor and the clip	Floor and clip first, then the exchange	Floor/clip changes went unpaid: energy and momentum not conserved
6	Self-gravitating profile rescaled linearly to the column density	Midplane density iterated until the column matches	A rescaled self-gravitating profile is not in equilibrium
7	g_{sg} = 0 at two cell centres, left-point sum	g_{sg} = 0 on the midplane face, trapezoid rule	Two midplane cells without self-gravity, first-order bias
8	Warning on the last allowed iteration even when converged	Warning only if the loop ran out	Spurious warnings
9	Typo `1vars(11)`, ambiguous comment, no shebang	Fixed	Namelist read error; clarity

Records

1 Star formation ran twice in the sf problem

BUG FIX

`problems/sf/initproblem.F90 · lines 120–134, 155, 270`

BEFORE

```
            use sndistr,        only: sum_dmass_stars, sfr_dump, sfr2hdf, sne_dump
            use star_formation, only: SF, dmass_stars
            ...
            call SF(forward, sfr_dump, sfr2hdf, sne_dump)
            if (forward) sum_dmass_stars = sum_dmass_stars + dmass_stars
      ...
      call init_user_module
      call init_SF
```

AFTER

```
            use sndistr,        only: sum_dmass_stars
            use star_formation, only: dmass_stars
            ...
            ! SF itself is called by the core (fluidupdate: SF_step); only accumulate the stellar mass
it formed
            if (forward) sum_dmass_stars = sum_dmass_stars + dmass_stars
      ...
      call init_user_module
      ! init_SF is called by the core (initpiernik)
```

WHY Star formation moved into the core: `initpiernik.F90:221` calls `init_SF`, and both `fluidupdate.F90:256` and `unsplit_fluidupdate.F90:184` call `SF_step`, which calls `SF`. The sf problem still made both calls from its own hooks, so every step formed stars and injected feedback twice and the SF module was initialised twice. The hook now only accumulates the mass the core's call formed.

2 Free-fall time with the stellar density

BUG FIX

`src/particles/star_formation.F90 · line 279`

BEFORE

```
tdyn = sqrt(3 * pi / (32 * newtong * cg%u(pfl%idn,i,j,k) + cg%q(ig)%arr(i,j,k)))
```

AFTER

```
tdyn = sqrt(3 * pi / (32 * newtong * (cg%u(pfl%idn,i,j,k) + cg%q(ig)%arr(i,j,k))))
```

WHY $t_{ff} = \sqrt{3\pi / 32\, G\, \rho_{tot}}$ with ρ_{tot} = gas + stars (`ig` is the mapped particle density `nbdn`). A missing pair of brackets added the stellar density without the factor 32 G = 0.144 (PSM units), so in cells holding stars it counted about 7 times too much and t_{dyn}, which enters the SF criterion, came out too short. Cells without stars were unaffected.

3 SN kinetic energy on the final cell state

BUG FIX

`src/particles/star_formation.F90 · TIGRESS_injection, lines 1099–1179`

BEFORE (both partly resolved and low-density branches)

```
      Ekin0 = ekin(cg%u(pfl%imx,i,j,k), cg%u(pfl%imy,i,j,k), cg%u(pfl%imz,i,j,k), cg%u(pfl%idn,i,j,k))
      v(:) = cg%u(pfl%imx:pfl%imz,i,j,k)/cg%u(pfl%idn,i,j,k)
      mp0cost = cg%u(pfl%idn,i,j,k) * (abs(ijk1(xdim)*v(xdim)) + abs(ijk1(ydim)*v(ydim)) +
abs(ijk1(zdim)*v(zdim))) / sqrt(real(aijk1))
      mp0sq   = cg%u(pfl%idn,i,j,k)**2 * (v(xdim)**2 + v(ydim)**2 + v(zdim)**2)
```

```
      mp = sqrt(max(0.0, mp0cost**2 + 2*(Ekin0 + en_SN09 * mfdv * 0.28 *frac) * (cg%u(pfl%idn,i,j,k)+
10.0 *frac / cg%dvol * merge(1,0, inject_mass)) - mp0sq)) - mp0cost
      padd = mp / sqrt(real(aijk1))
      ...                                    ! later: 10 Msun injected, region equalized (density and
momentum rescaled)
```

AFTER

```
   ekin_add = 0.0                          ! kinetic energy to add in this cell; padd is derived from it
below
   ...  ekin_add = en_SN09 * mfdv * 0.28 * frac      ! partly resolved: 28% kinetic, 72% thermal
   ...  ekin_add = en_SN09 * mfdv * frac             ! low density: all kinetic
   ...                                     ! mass injection and equalization as before
   if (ekin_add > 0.0) then
      ! Momentum mp along n = ijk1/|ijk1| that raises the kinetic energy of the final (mass-injected,
equalized) state by
      ! exactly ekin_add: |m + mp n|^2/(2 rho) - |m|^2/(2 rho) = ekin_add  =>  mp = -n.m + sqrt((n.m)^2
+ 2 rho ekin_add)
      mp0cost = dot_product(real(ijk1), cg%u(pfl%imx:pfl%imz,i,j,k)) / sqrt(real(aijk1))
      mp      = sqrt(mp0cost**2 + 2.0 * cg%u(pfl%idn,i,j,k) * ekin_add) - mp0cost
      padd    = mp / sqrt(real(aijk1))
   endif
```

WHY

The momentum kick was solved for the cell state before the 10 $M_\odot$ mass injection and before the injection region was equalised, which rescales density and momentum. It also mixed the old density in `mp0sq` with the new one in the bracket and used $|v_i|$ instead of the signed projection n·m. The kinetic energy actually added therefore differed from the 28% (or 100%) of E_{SN} it should be. After repeated injections into the same cell the argument of the square root could even go negative, which commit 1dd8091f8 hid with `max(0, …)`. Solving on the final state adds exactly `ekin_add`. With ρ > 0 and `ekin_add` ≥ 0 the argument cannot be negative, so the guard is gone. The fully unresolved branch (RM > 1), which injects the terminal momentum, is unchanged.

4 Seed-star masses use the SF rate

BUG FIX

src/particles/star_formation.F90 · seed_initial_stars, lines 1452, 1501–1503

BEFORE

```
                m_exp     = eps_sf * sqrt(newtong) * rho**1.5 * cg%dvol * seed_lookback
```

AFTER

```
                ! Expected stellar mass from the volumetric Schmidt law over the lookback window,
                ! with the same eps_sf * rho / t_ff rate as SF (t_ff = sqrt(3 pi / (32 G rho)))
                m_exp     = eps_sf / sqrt(3. * pi / (32. * newtong)) * rho**1.5 * cg%dvol *
seed_lookback
```

WHY

Seeds stand for the stars the SF law would have made over the lookback window. SF uses $\varepsilon\,\rho/t_{ff} = \varepsilon\,\sqrt{(32G/3\pi)}\,\rho^{3/2}$; the seeds used $\varepsilon\,\sqrt{G}\,\rho^{3/2}$, which is 1.84 times smaller. The production runs do not seed stars (they start from perturbation modes), so they are unaffected.

5 CR–gas exchange after the floor and the clip

BUG FIX

src/fluids/cosmicrays_streaming/streamingcr_source.F90 · lines 178–232

BEFORE (order of operations)

```
  implicit solve  -> newec, newf1..3
  feedback:  gas momentum -= (newf - f_old)/cred ;  gas energy -= (newec - ec)   (capped by available
e_int)
  floor:     newec = max(newec, smallescr)
  clip:      |newf| <= scr_causality_limit * newec
```

AFTER

```
  implicit solve  -> newec, newf1..3
  ! Floor Ec and clip Fc before the feedback, so that the gas exchanges exactly the final change of Ec
and Fc
  floor, clip
  feedback:  de_c = newec - ec, capped by e_int left after the momentum update
             if capped: newec = ec + de_c, and Fc is clipped again to the capped Ec (unless
scr_redo_on_violation)
             gas energy -= de_c ;  gas momentum -= (newf - f_old)/cred
  ! last resort, only reached when the gas had no internal energy left to pay for raising Ec to the
floor
  if (use_smallescr .and. newec < smallescr) newec = smallescr
```

WHY

b22c7360a made the exchange energy-conserving: the gas pays exactly ΔE_c and receives $-\Delta F_c/c_{red}^2$. But the floor and the causality clip ran *after* the exchange. Every time they fired, E_c or F_c changed without the gas paying, so total energy and momentum were again not conserved, and a capped exchange could leave $|F_c|$ above the causality limit for the capped E_c. Now the gas exchanges the final change. The available internal energy is evaluated with the momentum the gas will have after the kick, which is equivalent to the old order. Only if the gas cannot pay for raising E_c to the floor does the last-resort floor create energy, and that case is rare (section B).

6 Column density with self-gravity

BUG FIX

`src/gravity/hydrostatic.F90 · hydrostatic_zeq_coldens, lines 137–167`

BEFORE

```
      sdprof = 1.0
      call hydrostatic_zeq_densmid(iia, jja, sdprof, csim2, sd)
      dprof(:) = dprof(:) * coldens / sd
```

AFTER

```
      sdprof = 1.0
      call hydrostatic_zeq_densmid(iia, jja, sdprof, csim2, sd)
#ifdef SELF_GRAV
      if (flind%any_fluid_is_selfgrav()) then
         ! Self-gravity makes the profile nonlinear in d0, so it cannot be rescaled: iterate on d0
instead.
         ! With sd ~ d0**p, 1/2 <= p <= 1, the update d0 <- d0 * coldens / sd contracts the log-error by
(1 - p) per step.
         do it = 1, maxit
            if (abs(sd - coldens) <= 1e-5 * coldens) exit
            sdprof = sdprof * coldens / sd
            call hydrostatic_zeq_densmid(iia, jja, sdprof, csim2, sd)
         enddo
         if (it > maxit) call warn("[hydrostatic:hydrostatic_zeq_coldens] column density not matched
with self-gravity")
         return
      endif
#endif /* SELF_GRAV */
      dprof(:) = dprof(:) * coldens / sd
```

WHY

Without self-gravity the equilibrium profile is linear in the midplane density, so one solve followed by a rescale gives the requested column density. With self-gravity the gravity depends on the profile itself. The rescaled profile then carries more or less mass than the gravity it was computed with, so the initial disk is not in equilibrium and starts with a collapse or expansion transient. Iterating on d_0 converges geometrically, because $\Sigma \propto d_0^p$ with p between ½ (fully self-gravitating) and 1 (external gravity only). It stops at a relative error of 10^{-5} and warns if 100 iterations are not enough.

7 Self-gravity: midplane on a face, trapezoid rule

BUG FIX

`src/gravity/hydrostatic.F90` · `thermal_hydro_main` lines 245–260, `selfgrav_gauss_1d` lines 388–408

BEFORE

```
               sgprofs(ksmid+1) = 0.0
               do ksub = ksmid+1, nstot-1
                  sgprofs(ksub+1) = sgprofs(ksub) - fpi * newtong * dprofs(ksub) * dzs
               enddo
               sgprofs(ksmid) = 0.0
               do ksub = ksmid, 2, -1
                  sgprofs(ksub-1) = sgprofs(ksub) + fpi * newtong * dprofs(ksub) * dzs
```

AFTER

```
            ! g_sg = -4 pi G int_0^z rho dz': the midplane (g_sg = 0) is the face between ksmid and
ksmid+1, half a subcell
            ! from both centres, and the mass between neighbouring centres is integrated with the
trapezoid rule
               sgprofs(ksmid+1) = - half * fpi * newtong * dprofs(ksmid+1) * dzs
               do ksub = ksmid+1, nstot-1
                  sgprofs(ksub+1) = sgprofs(ksub) - half * fpi * newtong * (dprofs(ksub) +
dprofs(ksub+1)) * dzs
               enddo
               sgprofs(ksmid) = half * fpi * newtong * dprofs(ksmid) * dzs
               do ksub = ksmid, 2, -1
                  sgprofs(ksub-1) = sgprofs(ksub) + half * fpi * newtong * (dprofs(ksub) + dprofs(ksub-
1)) * dzs
```

WHY For a symmetric column the midplane z = 0 lies on the face between the two central subcells. Setting $g_{sg} = 0$ at both of their centres left these two cells without self-gravity. The left-point sum then integrated the mass between centres to first order only and lagged half a cell behind. Now $g_{sg} = -4\pi G \int_0^z \rho\, dz'$ is integrated from the face with the trapezoid rule, which is second order and antisymmetric about the midplane. `selfgrav_gauss_1d` gets the same treatment, so both solvers agree.

8 Convergence warnings

DIAGNOSTIC

`src/gravity/hydrostatic.F90` · lines 275, 364; unused `dz` removed (lines 74, 447)

BEFORE → AFTER

```
      if ((isub >= miter) .and. isub /= 1) then        ->   if (self_g .and. isub > miter) then
      if ((isub >= miter) .and. self_g) then           ->   if ((isub > miter) .and. self_g) then
```

WHY After `do isub = 1, miter` runs out, `isub = miter + 1`; after an `exit`, `isub ≤ miter`. The old `>=` test also warned when the solve converged on its last allowed iteration. In `hydrostatic_zeq_densmid` the warning is now limited to the self-gravitating case: without self-gravity, gravity does not depend on the profile and there is nothing to converge. Now the warning appears only when the loop ran out. The results do not change.

9 Parameter files and script

CONFIG

`problems/sf/problem.par:54 · problems/{stratified_box, stratified_box_thermal, tallbox}/problem.par · visual/h52paraview.py`

- `1vars(11) = 'sneh'` → `!vars(11) = 'sneh'`: the stray `1` broke the namelist read of the sf problem's OUTPUT_CONTROL.
- "For case 3 all 3 fits are bad" → "For prob 3 (cases 6 and 7) all 3 fits are bad": the warning is about `prob = 3`, which is cases 6 and 7, not about case 3.
- `#!/usr/bin/env python3` added to h52paraview.py so the script can be run directly.

B. Validation

Same source as the production runs. The `source_uncommitted.diff` saved in each production run directory is identical to this commit's diff of `src/` and `problems/`. Only the script shebang was added later.

TEST	SETUP	RESULT
Production C1	tallbox_sf, MHD + diffusive CR, self-gravity, SF + SN, unsplit, 48×48×192, 3 ranks	**0 → 134.0 Myr**, 26 195 steps, 1341 tsl rows, no NaN; stopped on purpose
Production D	as C1 with streaming CR (two-moment), restarted from 3 to 6 ranks at 98.9 Myr	**0 → 102.4 Myr**, 17 375 steps, 1024 tsl rows, no NaN; restart picked up exactly; stopped on purpose
Compile	`./setup sf | stratified_box | tallbox`	**all build**
Start-up	each of the three, 5 steps, 2 ranks	**all run**, no hydrostatic convergence warning, no NaN

Safeguards and floors in the production runs (summed over all tsl intervals):

COUNTER	C1 (134 MYR)	D (102 MYR)	READING
Energy added by internal-energy/temperature floors	31×10^{51} erg	27×10^{51} erg	about 0.5% and 0.6% of the injected SN energy (5 912 and about 4 190 SNe)
CR exchange capped by available e_{int} (cell-steps)	—	4.9×10^{5}	6×10^{-5} of all cell-steps
Face states needing the first-order fallback	0	1.3×10^{4}	about 0.8 face events per step (summed over RK stages and sweeps)
Last-resort E_c floor (`nfloor`)	—	0	never reached
Alfvén-speed ceiling: mass added	3 916 $M_\odot$	665 $M_\odot$	4×10^{-4} and 6×10^{-5} of the gas mass

C1 needed no face fallback at all and D fewer than one per step. The last-resort CR floor was never reached, and the floors added under 1% of the SN energy. So the reordered exchange (record 5) and the new injection (record 3) do not leave states the safeguards have to repair. Records 1, 4 and 6–8 act only on problems or code paths these two runs do not use (problems/sf, seed_initial_stars), or only at t = 0. For those, the start-up runs above check only that they compile and run. They have not been validated physically against a reference.

C. Open issues found in the production runs (not changed in this commit)

OPEN **1. The net magnetic flux leaves the box during the field-SN driving.** The box-averaged B_y (net flux, which in an x–y periodic box changes only through the z faces) falls from 0.79 μG to ≈ 0 between 15 and 50 Myr in both runs. Midplane |B| falls from 4.6 to 0.1–0.2 μG, so after 50 Myr the MHD runs are effectively hydro + CR. Budget for C1: the flux carried out through |z| = 2 kpc, $\int(v_z B_y - v_y B_z)dt$, accounts for the loss to within the 10 Myr snapshot sampling (B_y + outflow stays at 0.72–0.79). The interior update and the outd boundary therefore conserve flux. An undriven 40 Myr test with the same boundaries keeps the flux to 0.1%. The flux moves from the dense disk (99% of mass, 91% of flux at 10 Myr; 57% of mass, 2% of flux within |z| < 500 pc at 40 Myr) into the volume-filling hot gas (2.8% of mass, 60% of flux), and the $T \approx 3 \times 10^6$ K, $v_z \approx 300$–400 km s^{-1} wind carries it out. Whether the dense gas loses its field physically (draining along lifted field lines) or through grid-scale reconnection at 20.8 pc is still open: the 41.7 pc test makes no hot wind and cannot separate the two. Proposed next step: a 0.5 × 0.5 × 4 kpc column at 20.8 pc, with driving SNe at density peaks (frac_peak = 1) against the current placement.

OPEN **2. The two CR modules use different outflow boundaries.** The one-moment CRs (C1) set the ghost E_{CR} to `smallecr = 0` (`bnd_u`), which is an absorbing boundary: P_{CR} falls to ≈ 2 K cm^{-3} in the last cell and CRs drain by diffusion on $L^2/\kappa \approx 40$ Myr. The two-moment CRs (D) copy E_c and F_c from the edge cell (`bnd_scr`), a transmissive boundary, and P_{CR} stays flat (212 K cm^{-3}). With B ≈ 0, D's streaming speed is v_A ≈ 0.5 km s^{-1}, and D uses the same $\kappa_\parallel$, $\kappa_\perp$ as C1. So the higher CR pressure above 1 kpc in D (2.7× at 1.5 kpc at 90 Myr, 4× more CR energy overall) is mostly this boundary difference, not streaming. Proposed fix: zero-gradient CR ghosts in `bnd_u` for out/outd, matching `bnd_scr`.

OPEN **3. Cumulative SF counters are not saved in restart files.** After D's restart, `SNe_tot` and `dmass_stars_tot` started again from 0 (4025 → 0, 1.20 × 10^5 → 404 $M_\odot$), while `nSN_field` carried over. These are diagnostics only; the dashboard script now adds the previous total back at a restart.

D. State of the runs

RUN	STOPPED AT	NEWEST RESTART	NOTES
C1 (MHD + diffusive CR)	t = 134.0 Myr, step 26 195	tallbox_sf_C1c_0004.res	Star formation restarted at about 85 Myr, $\Sigma_{SFR} \approx 3$–8×10^{-3} $M_\odot$ yr^{-1} kpc^{-2}
D (MHD + streaming CR)	t = 102.4 Myr, step 17 375	tallbox_sf_D_s_0006.res	Restart with `runs/production_D6.sh` (6 ranks)

Both runs are worth continuing only after open issues 1 and 2 of section C are settled, because as they stand they compare two CR boundary conditions in a box that has lost its field, not streaming against diffusion.